Студент: Христина Ѓорѓиевска 233215

ИЗВЕШТАЈ ЗА ЛАБОРАТОРИСКА ВЕЖБА 2

1. ПОДГОТОВКА НА ОКОЛИНАТА И КОНФИГУРАЦИЈА

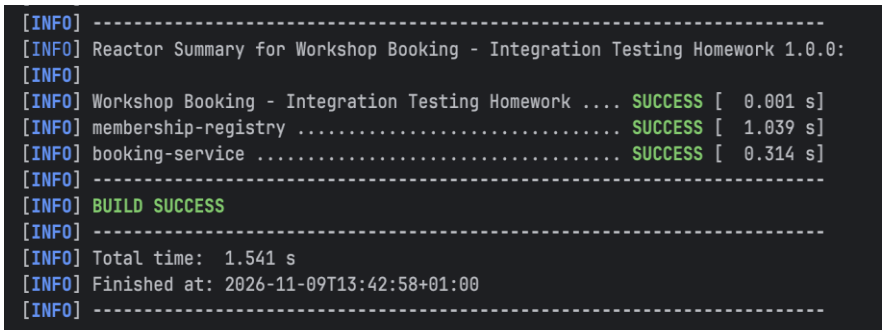
Поради тоа што во README.md беше наведено дека проектот ги има само зависностите потребни за апликацијата, требаше да додадам се што е потребно за тестовите.

1. **Зависности во booking-service/pom.xml**
   - сите зависности кои ги додадов се со test scope бидејќи се користат само за тестирањето на апликацијата.
   1. spring-boot-starter-webmvc-test: за стартување на целата апликација во тест (@SpringBootTest) и да се праќаат HTTP барања со MockMvc, без browser.
   2. spring-boot-starter-data-jpa-test: за тестирање на самите репозиториуми со вистинска база (@DataJpaTest).
   3. spring-boot-starter-restclient-test: за тестовите каде што апликацијата се стартува на вистинска порта и се праќаат вистински HTTP барања.

2. **Зависности во membership-registry/pom.xml**
   1. spring-boot-starter-webmvc-test: за стартување на целата апликација во тест (@SpringBootTest) и да се праќаат HTTP барања со MockMvc, без browser.
   2. spring-boot-starter-restclient-test: за тестовите каде што апликацијата се стартува на вистинска порта и се праќаат вистински HTTP барања.

3. **Plugins во booking-service/pom.xml и membership-registry/pom.xml**
   - во двата pom.xml го додадов ***maven-failsafe-plugin***. Со ова, unit тестовите и интеграциските тестови ќе се извршуваат одделно:
      - **sh mvnw test** - извршува unit тестови
      - **sh mvnw verify** - извршува интеграциски тестови

4. **Посебна конфигурација за тестовите (application-test.properties)**
   - Бидејќи апликацијата користи H2 база зачувана во ./data/bookings и очекува registry да работи на порта 9091, направив нова датотека src/test/resources/application-test.properties поради тоа што податоците во датотеката остануваат и после тестот, па следниот тест може да ги види, но исто така тестовите би зависеле од тоа дали registry-то е рачно стартувано.



2. ИДЕНТИФИКУВАЊЕ НА КОМПОНЕНТИ И НИВНИ МЕЃУСЕБНИ ЗАВИСНОСТИ

1. Проектот се состои од **две апликации**, од кои едната е главна: **booking-service** (за пријавување работилници - контролер, сервис, репозиториуми, базата и HTTP клиентот кон registry), а другата е помошна апликација: **membership-registry** (проверува дали некој член има право да се пријави).

Патот на едно барање за пријавување:
- Кога барањето за пријава пристигнува, BookingController-от го проследува до BookingService. Сервисот прво проверува дали податоците се валидни, потоа преку WorkshopRepository проверува дали работилницата постои и е отворена, па преку HttpMembershipGateway праќа барање до membership-registry за да провери дали членот кој го праќа барањето има право да се пријави на работилницаsa. На крај, пријавата се зачувува во база преку RegistrationRepository. Ако во некој од овие чекори се појави грешка, ApiErrors ја претвора во HTTP одговор со код и статус.
BookingService се наоѓа во средина и ги поврзува контролерот, базата и надворешното registry.

2. Кој од кого зависи:

Компонента	Од кого зависи
BookingController	BookingService
BookingService	WorkshopRepository, RegistrationRepository, MembershipGateway, Validator
Repository	База
HttpMembershipGateway	membership-registry (преку HTTP)
ApiErrors	Exceptions од сервис и база
MemberController	/

3. Bottom-up пристап:
- Прво ги тестирав долните слоеви, па потоа чекор по чекор додавав компоненти. Ги обрав

следните интеграции:
1. Репозиториуми со база (за проверка на зачувување, пребарување и ограничувања во база)
2. Сервис со валидатор, репозиториуми и база (за проверка за тоа дали се почитуваат правилата дефинирани во README.md и дали при јавување на грешка ништо не се зачувува во база)
3. Комуникацијата меѓу booking-service и membership-registry преку HTTP, од двете страни
4. Целиот тек од HTTP барање до база (за проверка на одговор и состојба во базата)
5. Трансакција при batch пријавување

Во тестовите, **границата што ја тестирам е секогаш реална, а замена користам само за надворешни зависности**.
- **Базата не ја заменувам** бидејќи со Mock не би се проверувале SQL-от и ограничувањата.
- Во тестот за сервисот, **MembershipGateway го заменувам со stub.** Во тестот за HTTP клиентот, registry-то го заменувам со локален HTTP сервер за да можам да симулирам грешки и бавен одговор, а HttpMembershipGateway е вистински.
- Вистинското registry го проверувам со посебен тест.

3. ИНТЕГРАЦИСКИ ТЕСТОВИ

1. RegistrationRepositoryIT.java

- Тестирање на меѓусебна работа на репозиториумите со базата.
- Реално: репозиториуми, Hibernate и H2.
- Замена: /

Тестови:
1. roundTrip(): пријавата се зачувува и кога ќе се прочита повторно, сите полиња се исти како и претходно.

```
25      @Test
26      void roundTrip(){
27          var workshop = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
28          var saved = registrationRepository.saveAndFlush(
29                  new Registration( reference: "REG-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 2, workshop));
30
31          testEntityManager.clear();
32
33          var loaded = registrationRepository.findByReference("REG-01").orElseThrow();
34
35          assertEquals(saved.getId(), loaded.getId());
36          assertEquals( expected: "Ana", loaded.getAttendee());
37          assertEquals( expected: "ana@example.com", loaded.getEmail());
38          assertEquals( expected: "C-01", loaded.getMemberId());
39          assertEquals( expected: 2, loaded.getTickets());
40          assertEquals( expected: "IT-01", loaded.getWorkshop().getCode());
41      }
```

2. unknownCodeReturnsEmpty(): за непостоечки код враќа empty.

```java
    @Test
    void unknownCodeReturnsEmpty(){
        assertTrue(workshopRepository.findByCode("not-created-workshop").isEmpty());
    }
```

3. unknownReferenceReturnsEmpty(): за непостоечки reference враќа empty.

```java
    @Test
    void unknownReferenceReturnsEmpty(){
        assertTrue(registrationRepository.findByReference("not-created-registration").isEmpty());
    }
```

4. findsByWorkshopAndAttendeePrefix(): пребарувањето по префикс ги враќа само “Ana...” од точната работилница, сортирано и со страници. “ana” не наоѓа ништо.

```java
    @Test
    void findsByWorkshopAndAttendeePrefix(){
        var java = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
        var other = workshopRepository.saveAndFlush(new Workshop( code: "IT-02",  title: "Other",  open: true));

        registrationRepository.saveAndFlush(new Registration( reference: "REG-01",  attendee: "Ana A",  email: "a1@example.com",  memberId: "C-01",  tickets: 1, java));
        registrationRepository.saveAndFlush(new Registration( reference: "REG-02",  attendee: "Ana B",  email: "a2@example.com",  memberId: "C-01",  tickets: 1, java));
        registrationRepository.saveAndFlush(new Registration( reference: "REG-03",  attendee: "Boris",  email: "b@example.com",  memberId: "C-01",  tickets: 1, java));
        registrationRepository.saveAndFlush(new Registration( reference: "REG-04",  attendee: "Ana C",  email: "a3@example.com",  memberId: "C-01",  tickets: 1, other));

        testEntityManager.clear();

        var page = registrationRepository.findByWorkshopCodeAndAttendeeStartingWith( code: "IT-01",  prefix: "Ana", PageRequest.of( pageNumber: 0,  pageSize: 1,
                Sort.by( ...properties: "attendee")));

        assertEquals( expected: 2, page.getTotalElements());
        assertEquals( expected: "REG-01", page.getContent().get(0).getReference());
        assertTrue(registrationRepository.findByWorkshopCodeAndAttendeeStartingWith( code: "IT-01",  prefix: "ana", PageRequest.of( pageNumber: 0,  pageSize: 5)).isEmpty());
    }
```

5. duplicateWorkshopCodeFails(), duplicateReferenceFails(), sameEmailForSameWorkshopFails(): овие тестови покажуваат дека базата одбива дупликати и фрла DataIntegrityViolationException.

```java
    @Test
    void duplicateWorkShopCodeFails(){
        workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));

        assertThrows(DataIntegrityViolationException.class, () -> workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Other",  open: true)));
    }

    @Test
    void duplicateReferenceFails(){
        var workshop = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));

        registrationRepository.saveAndFlush(new Registration( reference: "REG-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1, workshop));

        assertThrows(DataIntegrityViolationException.class, () -> registrationRepository.saveAndFlush(
                new Registration( reference: "REG-01",  attendee: "Boris",  email: "boris@example.com",  memberId: "C-01",  tickets: 1, workshop)));
    }

    @Test
    void sameEmailForSameWorkshopFails(){
        var workshop = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));

        registrationRepository.saveAndFlush(new Registration( reference: "REG-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1, workshop));
        assertThrows(DataIntegrityViolationException.class, () -> registrationRepository.saveAndFlush(
                new Registration( reference: "REG-02",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1,workshop )));
    }
```

6. sameEmailForDifferentWorkshopsIsAllowed(): ист email е дозволен за различни работилници.

```java
@Test
void sameEmailForDifferentWorkshopsIsAllowed(){
    var first = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
    var second = workshopRepository.saveAndFlush(new Workshop( code: "IT-02",  title: "Other",  open: true));

    registrationRepository.saveAndFlush(new Registration( reference: "REG-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1, first));
    registrationRepository.saveAndFlush(new Registration( reference: "REG-02",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1, second));

    testEntityManager.clear();
    assertTrue(registrationRepository.findByReference("REG-02").isPresent());
}
```

7. missingRequiredColumnFails(): базата не дозволува оставено празно задолжително поле.

```java
    @ParameterizedTest
    @CsvSource(value = {
            "null, Ana, ana@example.com, C-01",
            "REG-01, null, ana@example.com, C-01",
            "REG-01, Ana, null, C-01",
            "REG-01, Ana, ana@example.com, null"
    }, nullValues = "null")
    void missingRequiredColumnFails(String reference, String attendee, String email, String memberId){
        var workshop = workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
        assertThrows(DataIntegrityViolationException.class, () -> registrationRepository.saveAndFlush(
                new Registration(reference, attendee, email, memberId,  tickets: 1, workshop)));
    }
}
```

2. BookingServiceIT.java

- Тестирање на меѓусебна работа на сервисот, валидаторот, репозиториумите и базата.
- **Реално**: сервис, валидатор, репозиториуми, H2
- **Замена**: MembershipGateway

Тестови:

1. normalizesAndPersists(): сервисот го чисти името од празни места и го менува email-от во мали букви. Во базата се проверува дали е зачувано променетото.

```java
@Test  new *
void normalizesAndPersists(){
    workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));

    var result = bookingService.register(
            new BookingRequest( reference: "REG-01",  workshopCode: "IT-01",  attendee: "   Ana   ",  email: "Ana@Example.COM",  memberId: "C-01",  tickets: 2));

    testEntityManager.clear();

    var saved = registrationRepository.findByReference("REG-01").orElseThrow();

    assertEquals(result.id(), saved.getId());
    assertEquals( expected: "Ana", saved.getAttendee());
    assertEquals( expected: "ana@example.com", saved.getEmail());
    assertEquals( expected: "C-01", saved.getMemberId());
    assertEquals( expected: 2, saved.getTickets());
    assertEquals( expected: "IT-01", saved.getWorkshop().getCode());

}
```

2. ineligibleMemberFails(): member кој не смее да се пријави (MEMBER_INELIGIBLE), а во базата ништо не се зачувува.

```
    @Test  new *
    void ineligibleMemberFails(){
        workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
        var failure = assertThrows(BookingFailure.class, () -> bookingService.register(
                new BookingRequest( reference: "REG-01",  workshopCode: "IT-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-02",  tickets: 1)
        ));

        assertEquals( expected: "MEMBER_INELIGIBLE", failure.getCode());
        assertTrue(registrationRepository.findByReference("REG-01").isEmpty());
    }
```

3. sameEmailWithDifferentCaseFails(): ANA@example.com е дупликат на ana@example.com, бидејќи сервисот прво ја менува во мали букви, па потоа базата ја одбива.

```
    @Test  new *
    void sameEmailWithDifferentCaseFails(){
        workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));
        bookingService.register(new BookingRequest( reference: "REG-01",  workshopCode: "IT-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1));

        assertThrows(DataIntegrityViolationException.class, () -> bookingService.register(
                new BookingRequest( reference: "REG-02",  workshopCode: "IT-01",  attendee: "Ana",  email: "ANA@example.com",  memberId: "C-01",  tickets: 1)));
    }
```

4. findReturnsSavedRegistration(): find ја враќа зачуваната пријава заедно со кодот на работилницата.

```
    @Test  new *
    void findReturnsSavedRegistration(){
        workshopRepository.saveAndFlush(new Workshop( code: "IT-01",  title: "Java",  open: true));

        bookingService.register(new BookingRequest( reference: "REG-01",  workshopCode: "IT-01",  attendee: "  Ana  ",  email: "ana@example.com",  memberId: "C-01",  tickets: 3));

        testEntityManager.clear();

        var found = bookingService.find( reference: "REG-01");
        assertEquals( expected: "IT-01", found.workshopCode());
        assertEquals( expected: "Ana", found.attendee());
        assertEquals( expected: 3, found.tickets());
    }
}
```

3. HttpMembershipGatewayIT.java

- Тестирање на HTTP комуникацијата меѓу booking-service и registry, односно дали клиентот правилно го разбира добиениот одговор.
- **Реално**: HttpMembershipGateway, неговиот HTTP клиент и вистинско HTTP барање
- **Замена**: registry со локален HTTP сервер бидејќи вистинското не може да врати грешка, грешен JSON или бавен одговор.

Тестови:

1. readsEligibleMember(): серверот враќа исправен одговор.

```
    @Test  new *
    void readsEligibleMember(){
        status = 200;
        body = """
                {"memberId":"C-01","eligible":true}
                """;

        assertTrue(membershipGateway.isEligible( memberId: "C-01"));
    }
```

2. notFoundMeansMemberNotFound(): серверот враќа 404, MEMBER_NOT_FOUND.



3. brokenResponseIsContractError(): параметризиран тест со 4 неисправни одговори (недостасува поле, "true" како текст, друг memberId, не е JSON), REGISTRY_CONTRACT_ERROR. Со ова се проверува договорот за размена на податоци.



4. slowRegistryIsUnavailable(): серверот одговара по 2 секунди, а timeout е 500ms, REGISTRY_UNAVAILABLE.



4. BookingApiIT.java

- Тестирање на целиот тек: HTTP барање, контролер, сервис, репозиториуми, база.
- **Реално**: контролер, ApiErrors, сервис, валидатор, репозиториуми и база.
- **Замена**: MembershipGateway (stub)

Тестови:

1. createsRegistration(): враќа 201 и точен Location header, во базата името и email-от се зачувани изменети.

```java
@Test  new *
void createsRegistration() throws Exception {
    mvc.perform(post( uriTemplate: "/registrations").contentType("application/json").content("""
            {"reference":"REG-01","workshopCode":"IT-OPEN","attendee":" Ana ",
             "email":"Ana@Example.com","memberId":"C-01","tickets":2}
            """))
            .andExpect(status().isCreated())
            .andExpect(header().string( name: "Location",  value: "/registrations/REG-01"));
    em.flush(); em.clear();
    var saved = registrationRepository.findByReference("REG-01").orElseThrow();
    assertEquals( expected: "Ana", saved.getAttendee());
    assertEquals( expected: "ana@example.com", saved.getEmail());
}
```

2. rejectsAndSavesNothing(): параметризиран тест со 5 неуспешни случаи: 400 INVALID_INPUT, 404 WORKSHOP_NOT_FOUND, 409 WORKSHOP_CLOSED и 422 MEMBER_INELIGIBLE. Во сите случаи во базата ништо не е зачувано.

```java
@ParameterizedTest  new *
@CsvSource({
        "IT-OPEN,   C-01, 6, ana@example.com,     400, INVALID_INPUT",
        "IT-OPEN,   C-01, 1, ' ana@example.com ', 400, INVALID_INPUT",
        "NONE,      C-01, 1, ana@example.com,     404, WORKSHOP_NOT_FOUND",
        "IT-CLOSED, C-01, 1, ana@example.com,     409, WORKSHOP_CLOSED",
        "IT-OPEN,   C-02, 1, ana@example.com,     422, MEMBER_INELIGIBLE"
})
void rejectsAndSavesNothing(String workshop, String member, int tickets, String email, int status, String code) throws Exception {
    mvc.perform(post( uriTemplate: "/registrations").contentType("application/json").content("""
            {"reference":"REG-01","workshopCode":"%s","attendee":"Ana",
             "email":"%s","memberId":"%s","tickets":%d}
            """.formatted(workshop, email, member, tickets)))
            .andExpect(status().is(status))
            .andExpect(jsonPath( expression: "$.code").value(code));

    assertTrue(registrationRepository.findByReference("REG-01").isEmpty());
}
```

3. malformedJsonIsInvalidInput(): неисправен JSON враќа INVALID_INPUT:

```java
@Test  new *
void malformedJsonIsInvalidInput() throws Exception {
    mvc.perform(post( uriTemplate: "/registrations").contentType("application/json").content("broken json"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath( expression: "$.code").value( expectedValue: "INVALID_INPUT"));
}
```

4. duplicateReferenceIsConflict(): дупликат reference, 409 DATA_CONFLICT.

```java
@Test  new *
void duplicateReferenceIsConflict() throws Exception {
    var workshop = workshopRepository.findByCode("IT-OPEN").orElseThrow();
    registrationRepository.saveAndFlush(new Registration( reference: "REG-01",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 1, workshop));
    mvc.perform(post( uriTemplate: "/registrations").contentType("application/json").content("""
            {"reference":"REG-01","workshopCode":"IT-OPEN","attendee":"Boris",
             "email":"boris@example.com","memberId":"C-01","tickets":1}
            """))
            .andExpect(status().isConflict())
            .andExpect(jsonPath( expression: "$.code").value( expectedValue: "DATA_CONFLICT"));
}
```

5. returnsRegistrationContract(): GET враќа 200 со сите полиња и точни типови (id е број, reference е текст итн.).

```java
@Test  new *
void returnsRegistrationContract() throws Exception {
    var workshop = workshopRepository.findByCode("IT-OPEN").orElseThrow();
    registrationRepository.saveAndFlush(new Registration( reference: "REG-07",  attendee: "Ana",  email: "ana@example.com",  memberId: "C-01",  tickets: 2, workshop));
    entityManager.clear();
    mvc.perform(get( uriTemplate: "/registrations/REG-07"))
            .andExpect(status().isOk())
            .andExpect(jsonPath( expression: "$.id").isNumber())
            .andExpect(jsonPath( expression: "$.reference").value( expectedValue: "REG-07"))
            .andExpect(jsonPath( expression: "$.workshopCode").value( expectedValue: "IT-OPEN"))
            .andExpect(jsonPath( expression: "$.attendee").isString())
            .andExpect(jsonPath( expression: "$.email").isString())
            .andExpect(jsonPath( expression: "$.memberId").isString())
            .andExpect(jsonPath( expression: "$.tickets").value( expectedValue: 2));
}
```

6. unknownRegistrationIsNotFound(): непостоечка пријава, 404 REGISTRATION_NOT_FOUND.

```java
@Test  new *
void unknownRegistrationIsNotFound() throws Exception {
    mvc.perform(get( uriTemplate: "/registrations/NONE"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath( expression: "$.code").value( expectedValue: "REGISTRATION_NOT_FOUND"));
}
```

5. BatchTransactionIT.java

- Тестирање на целата трансакција при batch пријавување.
- **Реално**: цела апликација и база.
- **Замена**: MembershipGateway (stub што секогаш враќа true).

1. **Тестови**:

1.savesWholeBatchInOrder(): batch со 2 пријави, 201, ги враќа по ист редослед и двете се зачувани.

```java
@Test  new *
void savesWholeBatchInOrder() throws Exception {
    mvc.perform(post( uriTemplate: "/registrations/batch").contentType("application/json").content("""
            [{"reference":"REG-01","workshopCode":"IT-OPEN","attendee":"Ana",
              "email":"ana@example.com","memberId":"C-01","tickets":1},
             {"reference":"REG-02","workshopCode":"IT-OPEN","attendee":"Hristina",
              "email":"hristina@example.com","memberId":"C-01","tickets":1}]
            """))
            .andExpect(status().isCreated())
            .andExpect(jsonPath( expression: "$[0].reference").value( expectedValue: "REG-01"))
            .andExpect(jsonPath( expression: "$[1].reference").value( expectedValue: "REG-02"));
    assertTrue(registrationRepository.findByReference("REG-01").isPresent());
    assertTrue(registrationRepository.findByReference("REG-02").isPresent());
}
```

2. rollsBackWholeBatchWhenLaterItemFails(): втората пријава е дупликат на првата, 409. Во базата нема ниту една од двете, иако првата веќе била запишана. Постојната пријава REG-old останува непроменета.

```java
@Test  new *
void rollsBackWholeBatchWhenLaterItemFails() throws Exception {
    mvc.perform(post( uriTemplate: "/registrations/batch").contentType("application/json").content("""
            [{"reference":"REG-01","workshopCode":"IT-OPEN","attendee":"Ana",
              "email":"ana@example.com","memberId":"C-01","tickets":1},
             {"reference":"REG-02","workshopCode":"IT-OPEN","attendee":"Ana",
              "email":"ana@example.com","memberId":"C-01","tickets":1}]
            """))
            .andExpect(status().isConflict())
            .andExpect(jsonPath( expression: "$.code").value( expectedValue: "DATA_CONFLICT"));
    assertTrue(registrationRepository.findByReference("REG-01").isEmpty());
    assertTrue(registrationRepository.findByReference("REG-02").isEmpty());
    assertEquals( expected: "Old", registrationRepository.findByReference("REG-00").orElseThrow().getAttendee());
}
```

6. MemberProviderIT.java

- Тестирање на registry, дали навистина го враќа договорениот одговор.
- **Реално**: цела registry апликација и вистинско HTTP барање
- **Замена**: /

Тестови:

1. returnsKnownMember(): параметризиран: C-01 eligible: true, C-02 eligible: false.

```java
@ParameterizedTest  new *
@CsvSource({"C-01, true", "C-02, false"})
void returnsKnownMember(String memberId, boolean eligible) {
    client.get().uri( uri: "/members/" + memberId).exchange() ResponseSpec
            .expectStatus().isOk()
            .expectBody() BodyContentSpec
            .jsonPath( expression: "$.memberId").isEqualTo(memberId)
            .jsonPath( expression: "$.eligible").isEqualTo(eligible);
}
```

2. unknownMemberIsNotFound(): непостоечки член 404 и code: MEMBER_NOT_FOUND.

```java
@Test  new *
void unknownMemberIsNotFound() {
    client.get().uri( uri: "/members/C-99").exchange() ResponseSpec
            .expectStatus().isNotFound()
            .expectBody() BodyContentSpec
            .jsonPath( expression: "$.code").isEqualTo( expectedValue: "MEMBER_NOT_FOUND");
}
```

4. НЕЗАВИСНОСТ И ПОВТОРЛИВОСТ НА ТЕСТОВИТЕ

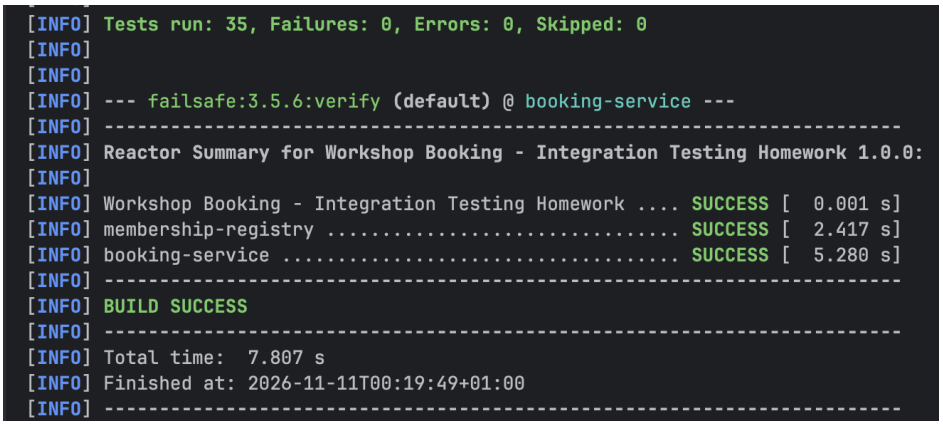
- Секој тест сам си креира податоци кои му се потребни за тестирање
- Базата е H2 (in-memory) и секогаш почнува празна
- По секој тест, податоците се бришат:

1. Кај RegistryRepositoryIT, BookingServiceIT и BookingApiIT - автоматски (rollback)
2. Кај BatchTransactionIT рачно (@AfterEach)
3. Кај HttpMembershipGatewayIT локалниот сервер ги гаси во @AfterEach

- Тестовите не користат фиксни порти и не зависат од рачно стартувани апликации.

5. ИЗВРШУВАЊЕ НА ТЕСТОВИТЕ

Класа	Број на тестови
RegistrationRepositoryIT	**12**
BookingServiceIT	**4**
HttpMembershipGatewayIT	**7**
BookingApiIT	**10**
BatchTransactionIT	**2**
MemberProviderIT	**3**
	38





6. НЕПРОВЕРЕНИ АСПЕКТИ

- Тестовите користат H2 (in-memory) па затоа не е проверено дали се работи исто и на PostgreSQL.
- Не е проверен foreign key помеѓу registration и workshop
- Грешките 502 и 503 не се проверени преку целиот API, туку само во HttpMembershipGatewayIT.
- Нема тест каде што booking-service и registry работат заедно.
- Не е проверено што се случува кога две барања истовремено се пријавуваат со ист email.